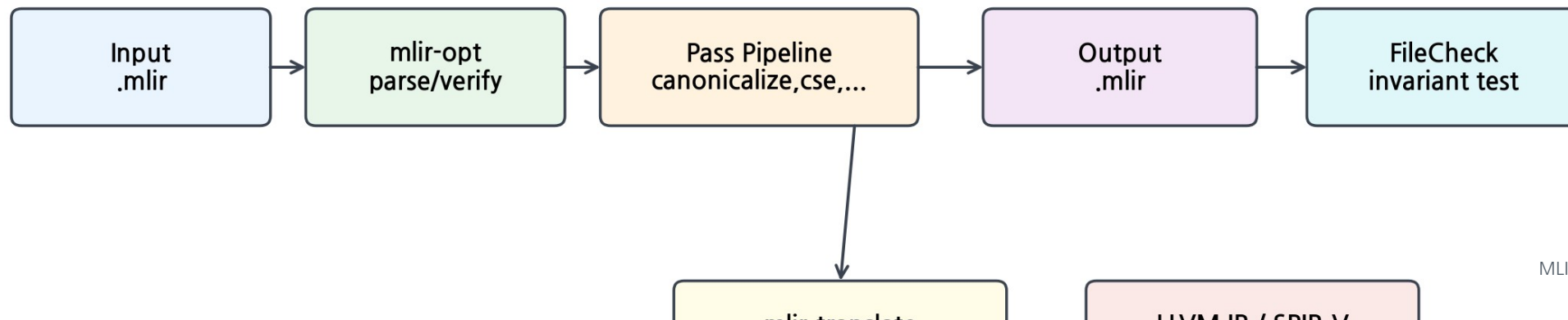


Lecture 04

mlir-opt, mlir-translate, FileCheck

Pass pipeline execution and regression testing for AI/NPU compiler development

NPU Compiler Tooling Loop: transform -> inspect -> test -> regress



- mlir-opt 로 MLIR input 을 parse/verify 하고 pass pipeline 을 실행한다 .
- textual pass pipeline 문법을 읽고 NPU lowering pipeline 을 command line 으로 표현한다 .
- mlir-translate 의 역할을 MLIR 내부 pass 와 구분한다 .
- FileCheck + lit RUN line 으로 regression test 를 작성한다 .
- NPU compiler 의 fusion/tiling/bufferization/codegen invariant 를 테스트로 만든다 .

4 강의 위치 : toolchain 을 몸에 익히는 시간

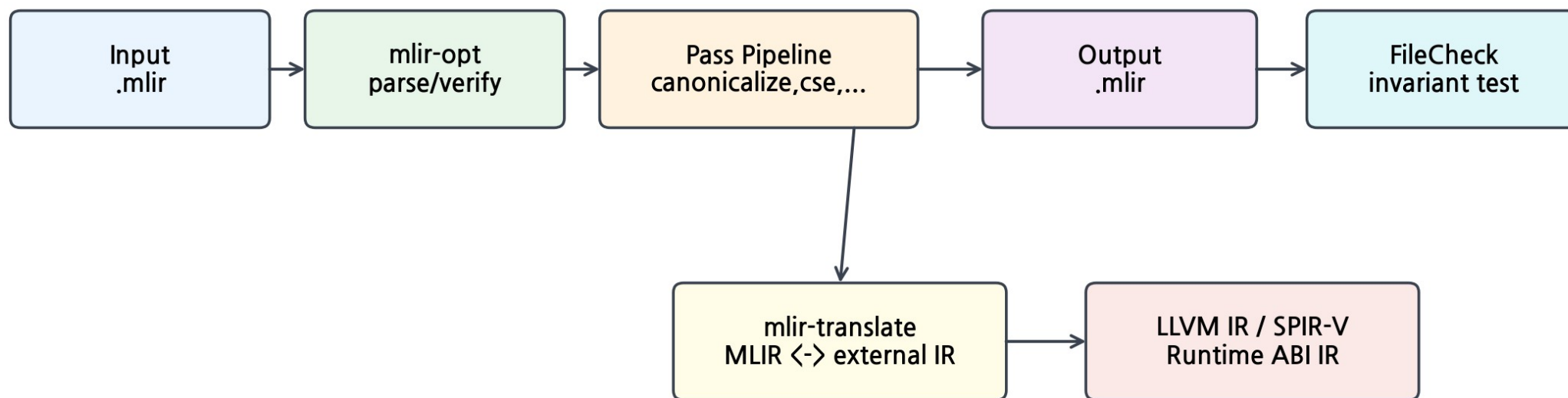
1-3 강 : IR 을 읽는 눈 - Operation, Value, Type, Attribute, Region, Block

4 강 : IR 을 실행하고 , 관찰하고 , 테스트하는 도구 사용법

5 강 이후 : Dialect/Pass/Rewrite/Conversion 을 만들 때 4 강 테스트 패턴을 계속 재사용

NPU compiler 개발에서는 pass 하나를 만들 때마다 최소 3 종 test 가 필요 : positive, negative, ordering/invariant

NPU Compiler Tooling Loop: transform -> inspect -> test -> regress



4강 목표: command line tool을 익혀 pass pipeline을 반복 실행하고, FileCheck로 깨지지 않는 회귀 테스트를 작성한다.

입력 textual/bytecode MLIR 을 in-memory IR 로 load 한다 .

Verifier 를 실행해 well-formed IR 인지 확인한다 .

선택된 pass 또는 pass pipeline 을 순서대로 실행한다 .

결과 IR 을 다시 textual MLIR 로 serialize 한다 .

주된 용도 : pass 개발 , lowering debug, regression test 생성

```
# parse + verify + print
```

```
mlir-opt input.mlir
```

```
# stdin/stdout 기반 사용
```

```
cat input.mlir | mlir-opt -o output.mlir
```

```
# generic form 으로 출력해 hidden details 확인
```

```
mlir-opt input.mlir --mlir-print-op-generic
```

CSE pass 실행

```
mlir-opt cse_test.mlir --pass-pipeline='builtin.module(func.func(cse))'
```

canonicalize + CSE

```
mlir-opt input.mlir --pass-pipeline='builtin.module(func.func(canonicalize,cse))'
```

핵심 : pass 이름뿐 아니라 어느 operation level 에 anchor 되는지도 중요하다 .

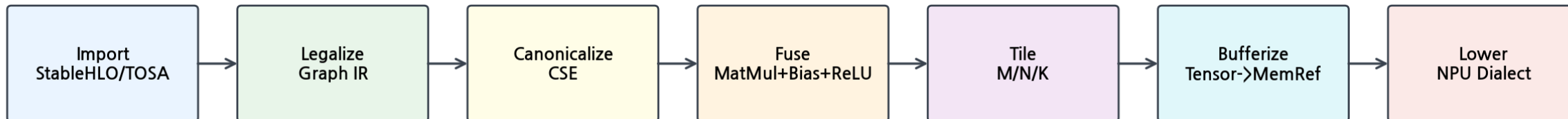
```
pipeline ::= op-anchor '(' pipeline-element (',' pipeline-element)* ')'  
  
builtin.module(  
  func.func(cse,canonicalize),  
  convert-func-to-llvm{use-bare-ptr-memref-call-conv=1}  
)
```

op-anchor: builtin.module, func.func, any 등 pass manager 실행 단위

nested pipeline: module pass 와 function pass 를 섞어 구성

options: pass 별 key=value 설정을 중괄호에 작성

NPU Lowering Pipeline을 mlir-opt pass-pipeline 문자열로 표현



Example: `builtin.module(func.func(canonicalize,cse), convert-..., npu-fuse, npu-tile{tile-m=16 tile-n=16 tile-k=32})`

Legalize 전에 fusion 을 시도하면 op semantics 가 아직 frontend dialect 에 흩어져 있을 수 있다 .
Tiling 전에 layout propagation 이 없으면 SRAM burst 와 bank conflict 가 잘못 계산된다 .
Bufferization 이후에는 tensor-level rewrite 가 어려워진다 .
DMA/barrier/compute ordering 은 기능 correctness 와 직접 연결된다 .
따라서 pass pipeline 은 compiler architecture document 의 실행 가능한 버전이어야 한다 .

- mlir-print-ir-before-all / --mlir-print-ir-after-all: 모든 pass 전후 IR dump
- mlir-print-ir-after-change: IR 이 실제로 바뀐 pass 이후만 dump
- mlir-print-ir-tree-dir=<dir>: pass 별 dump 를 directory tree 에 저장
- mlir-timing: pass 별 compiler latency 확인
- mlir-disable-threading: deterministic debug 와 local reproducer 에 유용

1. pass 가 실제로 IR 을 바꾸는지 확인

```
mlir-opt model.mlir --pass-pipeline="$PIPELINE" --mlir-print-ir-after-change
```

2. 대형 graph 는 파일 트리로 dump

```
mlir-opt model.mlir --pass-pipeline="$PIPELINE" --mlir-print-ir-tree-dir=/tmp/mlir_dump
```

3. pass latency 측정

```
mlir-opt model.mlir --pass-pipeline="$PIPELINE" --mlir-timing
```

mlir-translate 는 무엇이 다른가 ?

mlir-opt: MLIR 내부에서 pass 를 실행하고 MLIR 을 출력한다 .

mlir-translate: MLIR module 과 외부 representation 사이를 변환한다 .

대표 예 : LLVM dialect MLIR -> LLVM IR, SPIR-V binary round-trip

NPU 관점 : command buffer, firmware packet, simulator JSON 같은 외부 ABI emitter 설계와 유사한 책임

```
# LLVM dialect MLIR -> LLVM IR
```

```
mlir-translate --mlir-to-llvmir llvm_dialect.mlir > output.ll
```

```
# typical flow
```

```
high_level.mlir --mlir-opt lowering passes--> llvm_dialect.mlir --mlir-translate-->  
output.ll
```

compiler output 은 whitespace, SSA numbering, attribute order 가 자주 바뀐다 .

golden file 전체 비교는 유지보수가 어렵고 false failure 가 많다 .

FileCheck 는 필요한 pattern 과 invariant 만 확인한다 .

MLIR pass test 는 " 출력 전체 " 보다 "pass 가 보장해야 하는 성질 " 을 테스트해야 한다 .

FileCheck Test Anatomy

lit가 실행할 명령

```
// RUN: mlir-opt %s --pass-pipeline="builtin.module(func.func(cse))" | FileCheck %s
```

블록 분리용 anchor

```
// CHECK-LABEL: func.func @simple_constant
```

SSA 이름을 regex 변수로 capture

```
// CHECK-NEXT: %[[C:.*]] = arith.constant 1
```

같은 SSA value 재사용 검증

```
// CHECK-NEXT: return %[[C]], %[[C]]
```

입력 IR은 테스트 파일 자체에 포함

```
func.func @simple_constant() -> (i32, i32) { ... }
```



```
// RUN: mlir-opt %s -cse | FileCheck %s

// CHECK-LABEL: func.func @simple_constant
func.func @simple_constant() -> (i32, i32) {
  // CHECK-NEXT: %[RESULT:.*] = arith.constant 1
  // CHECK-NEXT: return %[RESULT], %[RESULT]
  %0 = arith.constant 1 : i32
  %1 = arith.constant 1 : i32
  return %0, %1 : i32, i32
}
```

CHECK-LABEL: 함수 /kernel 단위로 match 영역을 분리

CHECK: 순서 있는 positive match

CHECK-NEXT: 바로 다음 줄 . 남용하면 brittle 해짐

CHECK-SAME: 같은 줄의 type/attribute 일부 확인

CHECK-NOT: illegal op 가 사라졌는지 확인

CHECK-DAG: 순서 무관 match. scheduling-independent output 에 사용

%[[V:.*]]: SSA 이름 capture 로 printed name 변화에 대응

좋은 사용 : 독립 constant, metadata, 순서가 의미 없는 op 집합

위험한 사용 : npu.dma.load -> npu.barrier.await -> npu.matmul -> npu.dma.store

NPU command sequence 는 순서가 correctness 이다 .

DMA/store/barrier ordering 을 DAG 로 풀어버리면 실제 하드웨어 race bug 를 숨길 수 있다 .

Diagnostic test: negative case

```
// RUN: mlir-opt %s -split-input-file -verify-diagnostics

func.func @bad_branch() {
  cf.br ^missing // expected-error {{reference to an undefined block}}
}

// -----

func.func @bad_cmp(%a : f32) {
  // expected-error@+1 {{invalid predicate attribute specification: "foo"}}
  %result = arith.cmpf "foo", %a, %a : f32
  return
}
```

NPU Compiler Regression Test Matrix

Pass Stage	Positive Test	Negative Test	NPU Invariant
Legalize	supported op lowers	unsupported op emits error	no illegal graph op remains
Fuse	matmul+bias+relu fused	shape/layout mismatch not fused	activation order preserved
Tile	M/N/K tile loops appear	bad tile size rejected	tile covers full tensor
Bufferize	tensor -> memref	alias conflict diagnosed	memory space preserved
Codegen	npu.dma/npu.matmul emitted	missing barrier rejected	DMA before compute

```
cd examples  
mlir-opt 01_parse_verify.mlir  
mlir-opt 01_parse_verify.mlir | FileCheck 01_parse_verify.mlir
```

산출물 : input IR 이 well-formed 인지 확인하고 CHECK-LABEL 로 함수가 유지되는지 검증

```
mlir-opt 02_cse_regression_test.mlir --pass-pipeline='builtin.module(func.func(cse))' | FileCheck  
02_cse_regression_test.mlir
```

```
// CHECK-NEXT: %[[C:.*]] = arith.constant 1  
// CHECK-NEXT: return %[[C]], %[[C]]
```

실습 C: pass debug flags

```
mlir-opt 03_pass_pipeline_debug.mlir --pass-pipeline='builtin.module(func.func(cse,canonicalize))' --  
mlir-print-ir-after-all
```

```
mlir-opt 03_pass_pipeline_debug.mlir --pass-pipeline='builtin.module(func.func(cse,canonicalize))' --  
mlir-print-ir-after-change
```



```
// CHECK-LABEL: func.func @matmul_bias_relu  
// CHECK: npu.matmul  
// CHECK-SAME: tile_m = 16  
// CHECK-SAME: tile_n = 16  
// CHECK-SAME: tile_k = 32  
// CHECK: npu.relu  
// CHECK-NOT: stablehlo.dot_general
```

핵심 질문 : 각 CHECK line 은 어떤 NPU correctness invariant 를 검증하는가 ?

좋은 테스트 vs 나쁜 테스트

나쁜 테스트 : printer 가 출력한 모든 줄을 CHECK-NEXT 로 고정한다 .

좋은 테스트 : pass 가 보장해야 하는 핵심 invariant 만 확인한다 .

나쁜 테스트 : %0, %1 같은 SSA 이름에 의존한다 .

좋은 테스트 : %[[V:.*]] 같은 FileCheck 변수를 사용한다 .

나쁜 테스트 : negative case 없이 supported case 만 확인한다 .

좋은 테스트 : unsupported shape/layout/quantization diagnostic 도 테스트한다 .

Smoke: parse/verify, basic pass pipeline 실행

Transform: fusion/tiling/bufferization pass output invariant

Negative: unsupported op/layout/tile/quantization diagnostic

Ordering: DMA/barrier/compute/store command sequence

Translation: final command buffer 또는 LLVM IR/SPIR-V translation smoke

Performance sanity: pass timing regression threshold 또는 compile latency trend

Common failure modes

Pass 가 실행되지 않음 : op-anchor 가 잘못되었거나 dialect/pass registry 누락
Test 가 가끔 실패 : CHECK-DAG/parallel pass output/order assumption 문제
출력은 맞는데 FileCheck 실패 : CHECK line 이 너무 구체적
FileCheck 는 통과하지만 bug 존재 : 중요한 invariant 를 체크하지 않음
Negative test 실패 : diagnostic message 가 너무 불안정하거나 위치 지정이 틀림

mlir-opt 는 NPU compiler pass 개발의 기본 실험대이다 .
pass pipeline 문자열은 compiler lowering architecture 를 실행 가능한 형태로 표현한다 .
mlir-translate 는 MLIR 과 외부 IR/ABI representation 의 경계 도구이다 .
FileCheck 는 출력 전체가 아니라 compiler invariant 를 검증해야 한다 .
NPU 에서는 command order, memory space, layout, quantization metadata 가 test 대상이다 .

MLIR: Using mlir-opt

MLIR: Pass Infrastructure and Passes

MLIR: Testing Guide

LLVM: FileCheck Command Guide

MLIR source: mlir-translate main entry and translation infrastructure